

Vec2art Overview

Purpose and Vision

Vec2art is an open-source project that transforms raster images (JPEG/PNG/WebP) into expressive line-art SVG illustrations. The tool focuses on **high-performance vectorization** with hand-drawn aesthetics – it aims to process typical images in under 1.5 s. Unlike traditional vectorizers that simply trace edges, vec2art provides multiple algorithmic backends and an artistic enhancement pipeline to produce organic, hand-drawn results. The project has completed its first two phases (core line tracing and artistic enhancement) and is preparing for the frontend integration phase ¹.

Architecture and Technology Stack

Core Processing Engine

The heart of vec2art is a **Rust** codebase that implements line-tracing algorithms. Rust's performance, memory safety and rich ecosystem make it suitable for image processing. The engine uses SIMD optimizations and multi-threading via the `rayon` crate to achieve ultra-fast processing ². The code is compiled into different targets:

- `vectorize-core` – a pure Rust library implementing the algorithms and exposing a stable API; it contains modules for gradient analysis, background detection, dot placement and SVG generation. The dot-mapping development plan, for instance, defines gradient calculation, background detection and dot placement functions in Rust ³ ⁴ ⁵.
- `vectorize-cli` – a command-line binary that wraps the core library and exposes over 20 parameters for professional control ⁶. The CLI runs native on the host system and can be used to test backends, adjust detail levels, enable hand-drawn modes and run multi-pass tracing.
- `vectorize-wasm` – a WebAssembly (WASM) module built with `wasm-pack` for browser environments ⁷. WASM bindings expose the same vectorization functions to JavaScript/TypeScript and support multi-threading.

Compilation Target and Build Tools

The project uses `wasm-pack`, `cargo` and `wasm-opt` to compile Rust code to WebAssembly ⁸. SIMD and multi-threading support are enabled during compilation, and the `pkg/` output contains JavaScript bindings, the WebAssembly binary and TypeScript definitions ⁹. Build scripts (e.g., `rebuild-wasm.sh` and `rebuild-wasm.ps1`) automate the process.

Frontend (Phase 3)

The forthcoming frontend will be built with **SvelteKit 5**, **Tailwind CSS 4** and **TypeScript**, providing a real-time preview of the vectorized artwork ⁷. The WebAssembly module integrates with SvelteKit through helper functions that load the module and expose a `WasmVectorizer` instance. The SvelteKit integration

guide shows how to copy the compiled `pkg` directory into a Svelte project, configure Vite's cross-origin headers and use the vectorizer in a component ¹⁰ ¹¹. TypeScript definitions are provided for strong typing ¹².

Vectorization Backends and Algorithms

Vec2art offers four specialized backends, each optimized for different artistic styles ¹³:

1. **Edge backend** – default; uses optimized Canny edge detection with adaptive thresholds to produce clean, continuous line art ¹⁴. It supports multi-pass processing and hand-drawn effects.
2. **Dots backend** – generates stippling patterns by placing dots based on gradient analysis and adaptive density. The dot-mapping development guide outlines tasks for gradient calculation, background detection, dot placement and SVG rendering ³ ⁴ ⁵.
3. **Centerline backend** – extracts skeletons using the Zhang-Suen thinning algorithm to create single-pixel-width centerlines ¹⁵.
4. **Superpixel backend** – segments the image into regions with the SLIC algorithm and generates polygonal line art from the resulting superpixels ¹⁶.

These backends can operate in **single or multi-pass modes**. In multi-pass mode, the engine performs standard, reverse and diagonal passes to capture details from different scan directions ¹⁷. Users can choose to enable or disable reverse/diagonal passes depending on speed vs. quality considerations.

Artistic Enhancement Pipeline

To achieve a hand-drawn look, vec2art includes an **artistic enhancement pipeline**. Key features include variable stroke weights, tremor effects and line tapering ¹⁸. Users can select presets (e.g., clean, sketchy, artistic, expressive) or fine-tune individual parameters through the CLI or API. The `ConfigBuilder` in the WebAssembly module exposes functions for enabling multi-pass processing, adjusting tremor intensity, weight variation and tapering, selecting artistic presets and fine-tuning Canny thresholds ¹⁹.

Performance and Optimization

Performance is a core focus. The engine consistently achieves **<1.5s** processing times across all backends ²⁰. Optimizations include:

- **Multi-threading** via the `rayon` library to parallelize edge detection, path generation and other computations ²¹.
- **SIMD instructions** for image operations (available in modern browsers when SIMD is enabled) ² ²².
- **Memory efficiency** – buffer reuse reduces allocations ²³.
- **Configurable passes** – users can disable reverse or diagonal passes for speed or enable them for higher quality ¹⁷.

The engine automatically downscales large images to around 1080p to maintain performance and scales linearly with pixel count ²⁴. WASM builds include optional SIMD toggles for a ~2× speed-up ²².

CLI Usage Examples

The command-line interface demonstrates how to run different backends:

- Edge tracing: `cargo run --bin vectorize-cli -- trace-low input.png output.svg -- backend edge --detail 0.3` ²⁵.
- Dot stippling: `cargo run --bin vectorize-cli -- trace-low input.png output.svg -- backend dots --dot-density 0.1 --preserve-colors` ²⁶.
- Centerline skeletonization: `cargo run --bin vectorize-cli -- trace-low input.png output.svg --backend centerline --detail 0.4` ²⁷.
- Superpixel art: `cargo run --bin vectorize-cli -- trace-low input.png output.svg -- backend superpixel --detail 0.5` ²⁸.
- Hand-drawn mode: `--hand-drawn medium` toggles the artistic pipeline ²⁹.
- Multi-pass: `--multipass --enable-reverse --enable-diagonal` enables all passes ³⁰.

These examples illustrate the flexibility of the CLI and highlight how various options map to artistic styles.

WebAssembly API and Configuration

In the browser, developers interact with the WebAssembly module through a JavaScript/TypeScript API. The API exports a `process_image_to_svg()` function that converts an image buffer into an SVG string ³¹. Developers build configurations using the `ConfigBuilder` class to select backends, enable multi-pass processing, toggle hand-drawn effects, adjust thresholds, set maximum resolution and enable SIMD ¹⁹. The module also provides functions to retrieve processing statistics (e.g., processing time, backend used) ³².

A quick example of using the WASM module in the browser involves loading an image into a canvas, creating a configuration with the `ConfigBuilder`, and calling `process_image_to_svg()` to generate an SVG string that can be inserted into the DOM ³³. Node.js usage follows the same pattern using the `@vec2art/vectorize-wasm` package ³⁴.

Project Structure

The repository is organized to separate concerns and support multiple targets ³⁵:

vec2art/	
├─ wasm/	# Rust/WASM line tracing engine
│ └─ vectorize-core/	# Pure Rust algorithms and API
│ └─ vectorize-cli/	# Native CLI
│ └─ vectorize-wasm/	# WebAssembly bindings for browser
│ └─ scripts/	# Automated testing and build scripts
│ └─ examples/	# Test images and sample outputs
│ └─ docs/	# Technical documentation and research
└─ frontend/	# SvelteKit frontend (Phase 3)
└─ README.md	# Project overview ³⁵

The documentation inside `wasm/docs/` provides deeper insights into individual algorithms and development plans. For example, the **dot mapping development guide** describes the step-by-step plan to replace complex flow-based algorithms with a dot-based approach ³ and details the tasks for gradient analysis, background detection, dot placement and SVG rendering ⁴ ⁵.

Summary

Vec2art is a comprehensive line-tracing and artistic enhancement engine that uses **Rust**, **WebAssembly** and **SvelteKit** to deliver high-performance, expressive vector art. Its modular architecture supports multiple backends optimized for different artistic styles, and its CLI and API allow fine-grained control over the vectorization process. With ultra-fast processing times (<1.5s), hand-drawn aesthetics and a future SvelteKit frontend, vec2art is poised to become a versatile tool for artists, developers and designers.

¹ ² ⁶ ⁷ ⁸ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ²⁰ ²¹ ²³ ²⁵ ²⁶ ²⁷ ²⁸ ²⁹ ³⁰ ³⁵ [GitHub](#)

<https://github.com/robmcDonald5/vec2art/blob/564c203d3b55ca77b934a784a014d34180416271/README.md>

³ ⁴ ⁵ [GitHub](#)

https://github.com/robmcDonald5/vec2art/blob/564c203d3b55ca77b934a784a014d34180416271/wasm/docs/DOT_MAPPING_DEVELOPMENT.md

⁹ ¹⁰ ¹¹ ¹² [GitHub](#)

https://github.com/robmcDonald5/vec2art/blob/564c203d3b55ca77b934a784a014d34180416271/wasm/docs/SVELTEKIT_INTEGRATION.md

¹⁹ ²² ²⁴ ³¹ ³² ³³ ³⁴ [GitHub](#)

<https://github.com/robmcDonald5/vec2art/blob/564c203d3b55ca77b934a784a014d34180416271/wasm/vectorize-wasm/README.md>